MSM Reset Bug Fix Summary

Bug Description

A critical bug was discovered in the `msm_reset()` function where the `counted_as_leak` flags in allocation metadata were not being cleared when statistics were reset.

Problem Details

The Memory-Safety Maniac (MSM) profile tracks memory allocations and uses a `counted_as_leak` flag to prevent double-counting of memory leaks in statistics. When `msm_detect_leaks()` is called multiple times (e.g., during repeated report generation), this flag ensures each leak is only counted once.

The Bug: When `msm_reset()` was called to clear statistics and start fresh, it:
- ✅ Cleared the statistics structure (including `memory_leaks_detected` counter)
- ❌ **Did NOT clear the `counted_as_leak` flags** in existing allocation metadata

Impact: After calling `msm_reset()`, subsequent leak detection would:
1. Find the same leaked allocations
2. See that `counted_as_leak` is still `true`
3. Skip incrementing the statistics counter
4. Result in `memory_leaks_detected` staying at 0 despite leaks being present

Problem Scenario

```
1. Track allocations (leaks exist)
2. First leak detection:
   - Leaks found and counted
   - counted_as_leak = true for each leak
   - memory_leaks_detected = N

3. Call msm_reset():
   - memory_leaks_detected = 0
   - counted_as_leak flags REMAIN true (BUG!)

4. Second leak detection:
   - Same leaks found
   - counted_as_leak still true
   - Statistics NOT updated
   - memory_leaks_detected stays at 0 (WRONG!)
```

The Fix

Code Changes

File: `tools/crrss/msm/msm.c`
Function: `msm_reset()` (lines 517-561)

Added iteration through all tracked allocations to reset the `counted_as_leak` flag:

```
crrss_status_t msm_reset(msm_context_t* ctx) {
    if (!ctx || !ctx->initialized) return CRRSS_ERROR_NOT_INITIALIZED;

    pthread_mutex_lock(&ctx->context_lock);

    // Clear statistics
    pthread_mutex_lock(&ctx->stats_lock);
    memset(&ctx->stats, 0, sizeof(msm_statistics_t));
    clock_gettime(CLOCK_MONOTONIC, &ctx->stats.analysis_start_time);
    pthread_mutex_unlock(&ctx->stats_lock);

    // *** NEW CODE: Reset leak counting flags for all tracked allocations ***
    if (ctx->allocation_table) {
        for (int i = 0; i < MSM_HASH_TABLE_SIZE; i++) {
            pthread_mutex_lock(&ctx->allocation_table->locks[i]);

            hash_node_t* node = ctx->allocation_table->buckets[i];
            while (node) {
                allocation_metadata_t* meta = (allocation_metadata_t*)node->value;
                if (meta) {
                    meta->counted_as_leak = false;  // Reset the flag
                }
                node = node->next;
            }

            pthread_mutex_unlock(&ctx->allocation_table->locks[i]);
        }
    }
    // *** END NEW CODE ***

    // Clear issues
    pthread_mutex_lock(&ctx->issue_lock);
    // ... rest of function
```

Key Points

1. **Thread-Safe:** Uses existing per-bucket locks to safely iterate through the hash table
2. **Efficient:** Only resets flags, doesn't modify or free allocation structures
3. **Complete:** Ensures all allocations have their leak counting state reset
4. **Maintains Tracking:** Allocations remain tracked; only the statistics state is cleared

Test Coverage

New Tests Added

File: `tools/crrss/tests/test_msm.c`

Test 1: `test_reset_clears_leak_counting_flags()`

Comprehensive test that verifies the bug fix:

1. **Setup:** Track 3 allocations without freeing (creating leaks)
2. **First Detection:** Detect leaks → should count 3
3. **Verify:** Check that `memory_leaks_detected = 3`
4. **Second Detection:** Detect again → should NOT increment (double-counting prevention working)
5. **Verify:** Check that count stays at 3
6. **Reset:** Call `msm_reset()`

7. **Verify Reset:** Check that `memory_leaks_detected = 0`
8. **Third Detection (BUG FIX TEST):** Detect leaks again → should count 3 again
9. **Verify Fix:** Check that `memory_leaks_detected = 3` (proves flags were cleared)
10. **Fourth Detection:** Detect again → should NOT increment
11. **Verify:** Count stays at 3 (double-counting prevention still works)

Test 2: `test_reset_preserves_allocation_tracking()`

Ensures the fix doesn't break allocation tracking:

1. Track 2 allocations
2. Detect leaks (2 found)
3. Call `msm_reset()`
4. Verify allocations are still tracked and accessible
5. Verify allocations are not marked as freed
6. Detect leaks again (2 found)
7. Verify statistics are correctly rebuilt

Test Results

```
========================================
  MSM Test Suite
========================================

[TEST] MSM Initialization ... ✓ PASS
[TEST] MSM Invalid Initialization ... ✓ PASS
[TEST] MSM Reset ... ✓ PASS
[TEST] Allocation Tracking ... ✓ PASS
[TEST] Deallocation Tracking ... ✓ PASS
[TEST] Double-Free Detection ... ✓ PASS
[TEST] Pointer Tracking ... ✓ PASS
[TEST] Pointer Validation ... ✓ PASS
[TEST] Use-After-Free Detection ... ✓ PASS
[TEST] Use-After-Free Static Detection ... ✓ PASS
[TEST] Double-Free Static Detection ... ✓ PASS
[TEST] NULL-Check Analysis ... ✓ PASS
[TEST] Buffer Overflow Detection ... ✓ PASS
[TEST] Buffer Access Checking ... ✓ PASS
[TEST] Memory Leak Detection ... ✓ PASS
[TEST] Memory Leak Static Analysis ... ✓ PASS
[TEST] Leak Double-Counting Prevention ... ✓ PASS
[TEST] Leak Counting with Multiple Report Generations ... ✓ PASS
[TEST] MSM Reset Clears Leak Counting Flags ... ✓ PASS (NEW)
[TEST] MSM Reset Preserves Allocation Tracking ... ✓ PASS (NEW)
[TEST] Comprehensive File Analysis ... ✓ PASS
[TEST] Code Snippet Analysis ... ✓ PASS
[TEST] Statistics Generation ... ✓ PASS
[TEST] Report Generation ... ✓ PASS
[TEST] Safety Score Calculation ... ✓ PASS
[TEST] Issue Query by Type ... ✓ PASS
[TEST] Issue Query by Priority ... ✓ PASS
[TEST] Integration Setup ... ✓ PASS
[TEST] Utility Functions ... ✓ PASS

========================================
  Test Summary
========================================
  Total Tests:  29
  Passed:       29 (100.0%)
  Failed:       0
========================================
```

All 29 tests pass successfully! (Originally 27 tests, added 2 new tests)

Files Modified

1. tools/crrss/msm/msm.c

- **Modified:** `msm_reset()` function
- **Lines Changed:** Added 13 lines of code (lines 528-544)
- **Purpose:** Reset `counted_as_leak` flags for all tracked allocations

2. tools/crrss/tests/test_msm.c

- **Added:** 2 new test functions (130 lines total)
- `test_reset_clears_leak_counting_flags()` (75 lines)
- `test_reset_preserves_allocation_tracking()` (55 lines)

- **Modified:** Main test runner to call new tests
- **Purpose:** Comprehensive verification of bug fix

Commit Information

- **Branch:** feature/crrss-tooling-stage2
- **Commit Hash:** a30b2db
- **Commit Message:** `fix(crrss/msm): Reset leak counting flags when statistics are cleared`
- **Files Changed:** 2
- **Lines Added:** 155 insertions

PR Status

- **PR Number:** #170
- **Status:** Updated with bug fix
- **Branch:** feature/crrss-tooling-stage2 → main

Verification

The fix has been verified to:

✅ Correctly reset leak counting flags when `msm_reset()` is called
✅ Allow leak detection to work properly after reset
✅ Maintain double-counting prevention within a statistics cycle
✅ Preserve allocation tracking through reset operations
✅ Pass all 29 test cases (100% success rate)
✅ Maintain thread safety with proper locking

Impact Assessment

Before Fix

- ❌ Statistics reset followed by leak detection would show 0 leaks
- ❌ Report generation after reset would be incomplete
- ❌ Long-running analysis with periodic resets would lose leak data

After Fix

- ✅ Statistics can be properly reset and rebuilt
- ✅ Leak detection works correctly after reset
- ✅ Multiple analysis cycles with resets produce accurate results
- ✅ No performance impact (simple flag reset)
- ✅ Maintains all thread safety guarantees

Conclusion

The bug fix successfully resolves the issue where `msm_reset()` failed to clear the `counted_as_leak` flags, preventing accurate leak detection after statistics resets. The implementation is thread-safe, efficient, and fully tested with comprehensive test coverage demonstrating both the fix and that existing functionality remains intact.